

Learner's Space: Agentic AI

Assignment 3: MCP LandChain and LangGraph

Name : Rutuparn Mandar Ranade

Roll No. – 24B3982

Question 1: Build a ReAct Agent with LangChain

Build a ReAct agent using LangChain that can answer questions requiring tool use. Your agent must have at least 2 tools of your choice. For example: a calculator, a web search tool, a weather lookup, a currency converter, a Wikipedia fetcher, etc. (use DuckDuckGoSearchRun from langchain_community. It is a free drop-in for web search with no API key needed).

The agent should:

1. Accept a user query as a string input.
2. Reason through which tool(s) to call using the ReAct pattern (Thought → Action → Observation).
3. Return a final answer after one or more tool calls.

Ans: **Code:**

```
# 1. Update the import to use langchain_classic
from langchain_classic.agents import AgentExecutor, create_react_agent
from langchain_community.tools import DuckDuckGoSearchRun
from langchain_core.tools import tool
from langchain import hub
from langchain_openai import ChatOpenAI

# 2. Define our custom calculator tool
@tool
def calculator(expression: str) -> str:
    """Useful for answering math questions. Input should be a valid
    mathematical expression."""
```

```

try:

    # Note: eval() is used here for simplicity in a controlled
assignment environment.

    return str(eval(expression))

except Exception as e:

    return f"Error evaluating expression: {e}"

def run_react_agent():

    # 3. Initialize tools and the LLM

    search = DuckDuckGoSearchRun()

    tools = [search, calculator]

    llm = ChatOpenAI(temperature=0, model="gpt-4o-mini")

    # 4. Pull the standard ReAct prompt from the LangChain hub

    prompt = hub.pull("hwchase17/react")

    # 5. Construct the ReAct agent and the executor using the classic
imports

    agent = create_react_agent(llm, tools, prompt)

    agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True,
handle_parsing_errors=True)

    # 6. Define 3 queries that force the use of both tools

    queries = [

        "What is the current population of Canada?", # Forces Search

        "What is 456 multiplied by 987?", # Forces Calculator

        "Find the height of the Eiffel Tower in meters and multiply it by
15." # Forces Both

```

```

]

for q in queries:

    print(f"\n{'='*50}\nRunning query: {q}\n{'='*50}")

    # The verbose=True flag in the executor will print the Thought ->
    Action -> Observation trace

    agent_executor.invoke({"input": q})

if __name__ == "__main__":

    run_react_agent()

```

Tools Used:

1. Calculator Tools
2. Search Tools

Output:

Question 2: Build a LangGraph Workflow with Cycles

Build a LangGraph agent that answers a research question through an iterative loop. It must loop at least once before terminating. The workflow must have at least 3 nodes and use a conditional edge to decide whether to loop or stop.

A suggested (but not required) structure:

- generate node: LLM generates an answer or partial answer.
- evaluate node: LLM (or a simple heuristic) checks whether the answer is good enough (e.g. is it longer than N words? Does it contain certain keywords? Does it cite a source?)
- refine node: LLM improves the answer if evaluation fails.

Conditional edge: after evaluate, if the answer passes → END; if not → back to refine (loop). Define your AgentState as a TypedDict. Print the final state at the end.

Ans: Code

```

from typing import TypedDict
from langgraph.graph import StateGraph, END
from langchain_openai import ChatOpenAI

# 1. Define the AgentState
class AgentState(TypedDict):

```

```

    question: str
    answer: str
    feedback: str
    is_acceptable: bool
    iterations: int

llm = ChatOpenAI(temperature=0.7, model="gpt-4o-mini")

# 2. Node: Generate initial answer
def generate_node(state: AgentState):
    prompt = f"Answer the following research question briefly: {state['question']}"
    response = llm.invoke(prompt)
    return {"answer": response.content, "iterations": state.get("iterations", 0) + 1}

# 3. Node: Evaluate the answer
def evaluate_node(state: AgentState):
    answer = state.get("answer", "")
    word_count = len(answer.split())

    # Heuristic check: Answer must be detailed (> 30 words) and contain the word "research"
    if word_count > 30 and "research" in answer.lower():
        return {"is_acceptable": True, "feedback": "Pass"}
    else:
        return {
            "is_acceptable": False,
            "feedback": f"The answer is too short ({word_count} words) or missing the keyword 'research'. Please expand."
        }

# 4. Node: Refine the answer based on feedback
def refine_node(state: AgentState):
    prompt = f"""
    Question: {state['question']}
    Previous Answer: {state['answer']}
    Feedback: {state['feedback']}

    Please improve the answer to resolve the feedback.

```

```

"""
    response = llm.invoke(prompt)
    return {"answer": response.content, "iterations": state["iterations"]
+ 1}

# 5. Conditional Edge Logic
def route_evaluation(state: AgentState):
    if state["is_acceptable"]:
        return END
    else:
        return "refine"

def run_cyclic_graph():
    # Build the graph
    workflow = StateGraph(AgentState)
    workflow.add_node("generate", generate_node)
    workflow.add_node("evaluate", evaluate_node)
    workflow.add_node("refine", refine_node)

    workflow.set_entry_point("generate")
    workflow.add_edge("generate", "evaluate")
    workflow.add_conditional_edges("evaluate", route_evaluation, {END:
END, "refine": "refine"})
    workflow.add_edge("refine", "evaluate") # Close the loop

    app = workflow.compile()

    # Run the graph
    initial_input = {"question": "Explain the significance of quantum
computing.", "iterations": 0}
    final_state = app.invoke(initial_input)

    print("\n--- Final Agent State ---")
    for key, value in final_state.items():
        print(f"{key.capitalize()}: {value}\n")

if __name__ == "__main__":
    run_cyclic_graph()

```

Output:

Question 3: Cold Email Agent via Gmail MCP

Build a LangGraph agent that drafts and sends cold emails to professors or companies using the Gmail MCP server.

MCP setup: Use `langchain_mcp_adapters` to connect to Gmail's MCP server. A free Gmail MCP server is available at: `npx @gptsript-ai/gmail-mcp-server` Alternatively, you can use the stdio-based MCP approach with the Gmail MCP server. Configure it in your script via `MultiServerMCPCClient` (from `langchain_mcp_adapters`).

What the agent must do:

1. Accept as input: a recipient email, a recipient name, and a context string.
2. Use an LLM node to draft a personalised cold email based on the context (subject line + body).
3. Use a review node that checks the draft: if it is too generic or too short (your choice of threshold), regenerate. If it passes → send.
4. Call the Gmail MCP tool to actually send the email.

The LangGraph graph must have at least: draft → review → (conditional) → send / loop back to draft. Demonstrate by sending one cold email (to yourself or a test address is fine).

```
import asyncio

from typing import TypedDict

from langgraph.graph import StateGraph, END

from langchain_openai import ChatOpenAI

from langchain_mcp_adapters.client import MultiServerMCPCClient


class EmailState(TypedDict):

    recipient_email: str

    recipient_name: str

    context: str

    draft_subject: str

    draft_body: str
```

```

feedback: str

is_approved: bool

status: str

llm = ChatOpenAI(temperature=0.7, model="gpt-4o-mini")

# 1. Node: Draft the email
def draft_node(state: EmailState):

    if state.get("feedback"):

        prompt = f"""Rewrite this cold email based on the feedback.

Recipient: {state['recipient_name']}

Feedback: {state['feedback']}

Previous Draft: {state.get('draft_body')}

Format output exactly as:

SUBJECT: [Your Subject Line]

BODY: [Your Email Body]"""

    else:

        prompt = f"""Write a cold email to {state['recipient_name']}
regarding {state['context']}.

Format output exactly as:

SUBJECT: [Your Subject Line]

BODY: [Your Email Body]"""

    response = llm.invoke(prompt).content

    # Safely parse the subject and body using string splitting

```

```

try:

    parts = response.split("BODY:")

    subject = parts[0].replace("SUBJECT:", "").strip()

    body = parts[1].strip()

    return {"draft_subject": subject, "draft_body": body}

except Exception:

    return {"draft_subject": "Connecting", "draft_body": response}

# 2. Node: Review the draft
def review_node(state: EmailState):

    body = state.get("draft_body", "")

    # Threshold check: Must be longer than 30 words
    if len(body.split()) > 30:

        return {"is_approved": True, "feedback": "Approved"}

    else:

        return {"is_approved": False, "feedback": "The email is too short. Please add more personalized context."}

def route_email(state: EmailState):

    return "send" if state["is_approved"] else "draft"

# 3. Node: Send via Gmail MCP
async def send_node(state: EmailState):

    print("\n[Connecting to Gmail MCP Server...]")

    # Initialize the stdio-based MCP client

    client = MultiServerMCPClient({

```

```

        "gmail": {
            "command": "npx",
            "args": ["-y", "@gptsript-ai/gmail-mcp-server"],
            "transport": "stdio"
        }
    })

tools = await client.get_tools()

# Locate the sending tool from the MCP server
send_tool = next((t for t in tools if "send" in t.name.lower()), None)

if send_tool:
    print(f"\n[Sending Email to {state['recipient_email']}]")

    await send_tool.ainvoke({
        "to": state["recipient_email"],
        "subject": state["draft_subject"],
        "body": state["draft_body"]
    })

    return {"status": "sent"}
else:
    print("\n[Error: Could not find send tool on MCP server]")

    return {"status": "failed"}

# 4. Build and run the asynchronous graph
async def run_email_agent():
    workflow = StateGraph(EmailState)

```

```

workflow.add_node("draft", draft_node)

workflow.add_node("review", review_node)

workflow.add_node("send", send_node)


workflow.set_entry_point("draft")

workflow.add_edge("draft", "review")

workflow.add_conditional_edges("review", route_email, {"send": "send",
"draft": "draft"})

workflow.add_edge("send", END)


app = workflow.compile()


test_input = {

    "recipient_email": "test@example.com", # Replace with your email
for testing

    "recipient_name": "Professor Smith",

    "context": "Expressing interest in joining their AI research lab
as a graduate student next fall."

}


# We use ainvoke because the MCP send node uses async/await

final_state = await app.ainvoke(test_input)

print("\n--- Final Status ---")

print(final_state["status"])


if __name__ == "__main__":

    asyncio.run(run_email_agent())

```

Output: